

Lecture 1: File System Abstractions

Overview

- A file system provides a logical view of persistent data stored on secondary storage.
- It hides device-specific details such as sectors, tracks, flash pages and controller commands.
- It provides naming, organization, protection, sharing and persistent access to data.
- Main abstractions: **files**, **directories**, **paths** and **file descriptors**.

Learning Objectives

- Explain why operating systems need a file-system abstraction.
- Describe the file abstraction and common file attributes.
- Explain fundamental file operations and file descriptors.
- Distinguish absolute and relative pathnames.
- Explain directories as an organizational abstraction.

Why Do We Need a File System?

Without a file system

- Applications would need to understand storage-device layout.
- Data would be difficult to name and locate.
- Protection and sharing would be difficult to implement.

With a file system

- Files provide named persistent objects.
- Directories organize files.
- The OS manages allocation, protection and consistency.

File Abstraction

- A **file** is a named logical container for related information.
- The OS maps the logical file to physical storage blocks.
- Applications normally do not need to know where those blocks reside.
- Typical file types:
 - regular files – text, executable, image, database, etc.
 - directories
 - device/special files
 - symbolic links

File Attributes / Metadata

- Name
- Identifier or inode number
- Type
- Size
- Location information
- Owner and group
- Protection/permissions
- Creation, modification and access timestamps
- Link count and other file-system-specific metadata

Basic File Operations

Operation	Purpose
create	create a new file
open	establish an access context
read	transfer data from file to process
write	transfer data from process to file
seek	change current file offset
close	release an open-file reference
delete	remove a directory entry/file
rename	change the file name/path

Open() and File Descriptors

- A process requests an `open()` operation using a pathname.
- The OS resolves the pathname and locates the file's metadata.
- The OS creates/updates an open-file-table entry.
- The process receives a small integer called a **file descriptor**.
- Subsequent `read()`, `write()`, `seek()` and `close()` operations can use this descriptor.

pathname → file metadata → open-file entry → file descriptor

Directory Abstraction

- A directory maps names to file-system objects.
- Directories provide hierarchical organization.
- The root directory is the starting point of the hierarchy.
- A directory may contain files and subdirectories.
- Typical operations: create, delete, list, rename and search.

`/home/faculty/os/lecture1.pdf`

Absolute vs. Relative Paths

Absolute path

- Starts from the root.
- Example: `/home/user/data/a.txt`

Relative path

- Interpreted relative to the current working directory.
- Example: `data/a.txt`

Special names: `.` = current directory, `..` = parent directory.

- **Silberschatz, Galvin & Gagne, Operating System Concepts, 10th Ed.**
 - Chapter 13 – File-System Interface
 - File concept, access methods, directory and disk structure, file-system mounting, sharing and protection
- **Stallings, Operating Systems: Internals and Design Principles, 9th Ed.**
 - Chapter 18 – File Management
 - File organization/access, directories, file sharing and Unix/Linux file management

Lecture Summary

- File systems provide a logical abstraction over physical storage.
- Files expose data and metadata through a controlled interface.
- Directories provide hierarchical naming.
- File descriptors connect processes to open files.
- These abstractions form the foundation for file-system implementation.

Think: What information must the OS maintain after a process executes `open("data.txt")`?

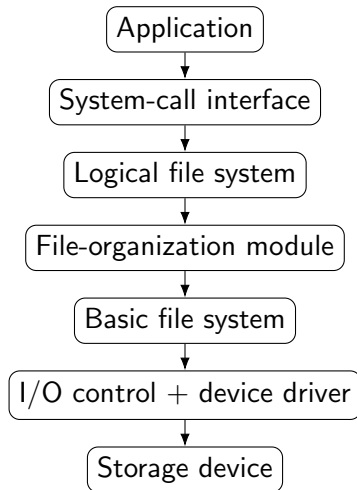
Lecture 2: File-System Organization and Implementation

- File systems contain both **on-disk** and **in-memory** data structures.
- The implementation is commonly organized in layers.
- Metadata structures allow the OS to translate logical file operations into physical block operations.

Learning Objectives

- Identify the major layers of a file-system implementation.
- Explain superblocks, inodes/FCBs, directories and data blocks.
- Distinguish on-disk and in-memory structures.
- Explain mounting and open-file management.

Layered File-System Architecture



Major On-Disk Structures

- **Boot block:** information needed to boot from the volume.
- **Superblock / volume control block:** file-system type, size, block information and state.
- **Free-space structure:** records available blocks/extents.
- **Directory structures:** map names to file identifiers.
- **File metadata:** inode/FCB-like structures.
- **Data blocks:** contain file contents.

Inode / File Control Block

- A file's metadata is stored in a structure such as an inode or FCB.
- Typical fields:
 - file type and permissions
 - owner/group
 - size
 - timestamps
 - link count
 - pointers/extents identifying data blocks
- The directory normally stores the name-to-file-identifier association.

Directory Entry vs. File Metadata

Directory	File metadata
filename	type
file identifier/inode number	size
pathname relationship	owner/group
	permissions
	timestamps
	data-block mapping

Separating names from metadata allows multiple directory entries to reference the same file object.

In-Memory Structures

- Mount table
- Buffer/page cache
- In-memory inode/FCB structures
- System-wide open-file table
- Per-process file-descriptor table
- These structures reduce repeated disk accesses and maintain the state of active file operations.

Mounting a File System

- 1 Identify a storage volume or file-system instance.
- 2 Read its file-system metadata.
- 3 Validate and initialize in-memory structures.
- 4 Attach the file system to a directory in the existing namespace.

/ + /data → mounted file system

From read() to Storage

- 1 Process invokes `read(fd, ...)`.
- 2 File descriptor identifies an open-file entry.
- 3 File metadata determines logical file offset and block mapping.
- 4 Logical block is translated to a physical block/storage location.
- 5 Cache is checked; if necessary, an I/O request is issued.
- 6 Data is copied/returned to the process.

- **Galvin, 10th Ed.:** Chapter 14 – File-System Implementation; file-system structure, implementation, directory implementation, allocation and free-space management.
- **Stallings, 9th Ed.:** Chapter 18 – File Management; Unix/Linux file management, secondary-storage management and file-system organization.

Lecture Summary

- File-system implementation is layered.
- Metadata structures connect names to physical data.
- On-disk structures provide persistence; in-memory structures provide efficient active access.
- Mounting integrates a file system into a namespace.

Lecture 3: File Allocation and Directory Implementation

- The allocation method determines how file blocks are placed on storage.
- Major classical methods: contiguous, linked and indexed allocation.
- Modern systems often use extents or extent-like structures.

Learning Objectives

- Explain contiguous, linked and indexed allocation.
- Analyze internal/external fragmentation and file-growth issues.
- Explain FAT and multi-level indexing.
- Compare allocation methods for different workloads.
- Describe linear-list and hash-table directory implementations.

Contiguous Allocation

- A file occupies consecutive disk blocks.
- Metadata can store starting block and length.

$$\text{block}(i) = \text{start} + i$$

- Excellent sequential and direct-access performance.
- Main problems: external fragmentation and difficulty growing a file.

Contiguous Allocation Example

Block	10	11	12	13	14	15	16	17	18	19
File A	A	A	A	A	A	A	-	-	-	-

If File A grows beyond block 15, adjacent blocks may be unavailable. The file may need relocation or an additional extent.

Linked Allocation

- File blocks can be located anywhere.
- Each block contains a pointer to the next block.

$$B_7 \rightarrow B_{19} \rightarrow B_4 \rightarrow B_{31}$$

- No external fragmentation caused by requiring consecutive blocks.
- Good for sequential access.
- Poor random access because blocks must be followed sequentially.
- Pointer overhead and reliability concerns are important.

FAT: File Allocation Table

- FAT stores the linked-list information in a table rather than inside every data block.
- A directory entry identifies the first block.
- The FAT entry for each block identifies the next block.
- End-of-file is represented by a special marker.
- Centralizing pointers makes traversal easier but the table can be large.

Indexed Allocation

- An index block contains pointers to the file's data blocks.
- Direct access is easier than linked allocation.

Index block $\rightarrow$ 7, 19, 4, 31, 42, ...

- Index-block overhead can be significant for small files.
- Large files can use multi-level indexing.

Multi-Level Indexing

- Single-level indexing may not provide enough pointers for large files.
- Multi-level schemes use indirect blocks.

`inode` $\rightarrow$ single indirect $\rightarrow$ data blocks

- Double and triple indirect structures extend the addressable file size.

Extent-Based Allocation

- An **extent** represents a contiguous range of blocks.
- A file can be represented using multiple extents.

$$\text{File} = \{(\text{start}_1, \text{length}_1), (\text{start}_2, \text{length}_2), \dots\}$$

- Reduces metadata compared with block-by-block mapping.
- Retains good sequential performance.
- Widely useful for large files.

Directory Implementation

Linear list

- Store filename and identifier pairs.
- Simple.
- Search may be $O(n)$.

Hash table

- Hash filename to a bucket.
- Faster average lookup.
- Requires collision handling.

Comparison

Method	Strength	Limitation
Contiguous	Excellent sequential/direct access	External fragmentation, growth
Linked	Easy growth	Poor random access
Indexed	Direct access	Index overhead
Extents	Good performance + compact mapping	Fragmentation can create more extents

- **Galvin, 10th Ed.:** Chapter 14 – Allocation Methods and Directory Implementation.
- **Stallings, 9th Ed.:** Chapter 18 – File Organization and Access; File Directories; Unix/Linux file management.

- No allocation method is optimal for every workload.
- Contiguous allocation favors performance but suffers from growth/fragmentation.
- Linked allocation favors sequential access and flexible growth.
- Indexed allocation supports direct access but consumes metadata space.
- Extents combine contiguous runs with flexible multi-extent files.